JavaScript Interface for Android Webview

ANDROID

Updated 31 January 2024

Save

In this article, we are going to learn how we can call android functions using javascript and vice versa using the javascript interface for android webview.

To know about webview and how it works, head over here.

For javascript to work in our webview, we need to enable it using webview settings.

```
Java:
webView.getSettings().setJavaScriptEnabled(true);

Kotlin:
webView.settings.javaScriptEnabled = true
```

Binding JavaScript To Android

Through an interface, WebView enables us to connect JavaScript code to Android code.

The class that provides the interface for JS must be passed, along with the name that will be used to show the instance in JS (for example, "Android").

As a result, an interface called **Android** will be created for JavaScript that is running in WebView.

By using the interface we can achieve the following:

- Run the methods described in Java/Kotlin from JS
- Run the JS methods from Java/Kotlin

Calling Java/Kotlin Method From JavaScript

Create a JavaScript Interface Class

```
class MyJsInterface(private val mContext: Context) {
        @JavascriptInterface
        fun getAndroidVersion(): Int {
            return Build.VERSION.SDK_INT
        }

        @JavascriptInterface
        fun showToast() {
            Toast.makeText(mContext, "Toast from android", Toast.LENGTH_SHO
RT).show()
        }
    }
```

Note: **@JavascriptInterface** annotation is required for **API 17** and above.

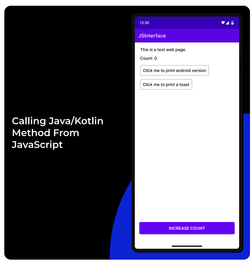
Then install this interface using the following code.

```
Java:
webView.addJavascriptInterface(new MainActivity.MyJsInterface(this), "Androi
d")

Kotlin:
webView.addJavascriptInterface(MyJsInterface(this), "Android")
```

Let's create an HTML file where we will use our Java/Kotlin functions

```
    label {
      display: block;
      padding: 8px 0;
    }
  </style>

  <body>
    <label>This is a test web page.</label>

    <label>Count: <span id="count">0</span></label>

    <label id="version"></label>

    <button onclick="printAndroidVersion()">
      Click me to print android version
    </button>
    <button onclick="showToast()">Click me to print a toast</button>
  </body>

  <script>
    const printAndroidVersion = () => {
      const version = Android.getAndroidVersion();
      document.getElementById(
        "version"
      ).innerText = `Android version ${version}`;
    };

    const showToast = () => {
      Android.showToast();
    };

    const increaseCount = () => {
      const count = document.getElementById("count").textContent;
      document.getElementById("count").textContent = Number.parseInt(count) + 1;
    };
  </script>
</html>
```

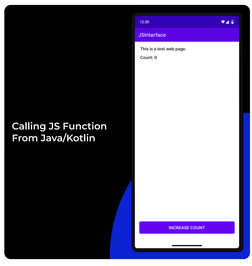
Output:



Calling JS Function from Java/Kotlin

```
        findViewById<Button>(R.id.button).apply {
            setOnClickListener {
                // Calling JS function from Android
                // This name should match the name of the method that
                // you have declared in your JS file
                webView.loadUrl("javascript:increaseCount()")
            }
        }
```

Output:



MainActivity's code

```
class MainActivity : AppCompatActivity() {
    private lateinit var mWebView: WebView

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)

        mWebView = findViewById<WebView>(R.id.webview).apply {  this: WebView
            loadUrl( url: "file:///android_asset/index.html")
            webViewClient = WebViewClient()
            addJavascriptInterface(MyJsInterface( mContext: this@MainActivity),  name: "Android")
            settings.javaScriptEnabled = true
        }

        findViewById<Button>(R.id.button).apply {  this: Button
            setOnClickListener {  it: View!
                // Calling JS function from Android
                // This name should match the name of the method that
                // you have declared in your JS file
                mWebView.loadUrl( url: "javascript:increaseCount()")
            }
        }
    }


    class MyJsInterface(private val mContext: Context) {
        @JavascriptInterface
        fun getAndroidVersion(): Int {
            return Build.VERSION.SDK_INT
        }

        @JavascriptInterface
        fun showToast() {
            Toast.makeText(mContext,  text: "Toast from android",  Toast.LENGTH_SHORT).show()
        }
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    xmlns:tools="http://schemas.android.com/tools"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    tools:context=".MainActivity">

    <WebView
        android:id="@+id/webview"
        android:layout_width="match_parent"
        android:layout_height="0dp"
        app:layout_constraintBottom_toTopOf="@id/button"
        app:layout_constraintTop_toTopOf="parent" />

    <Button
        android:id="@+id/button"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_margin="10dp"
        android:text="Increase count"
        app:layout_constraintBottom_toBottomOf="parent"
        app:layout_constraintEnd_toEndOf="parent"
        app:layout_constraintStart_toStartOf="parent" />

</androidx.constraintlayout.widget.ConstraintLayout>
```

Conclusion

This is it, hope you have understood how we can use an interface to communicate with our JS code and vice versa.

. . .

© Copyright 2010-2024, Webkul Software (Registered in India/USA). All rights reserved.

We use cookies to personalize your experience. By continuing to visit this website you agree to our use of cookies. Learn more about privacy policy
GOT IT!